

Seminario_Algoritmos

October 9, 2026

1 Algoritmos de optimización - Seminario

Nombre y Apellidos: - Gonzalo Domínguez Sandoya
- Adrián García Sánchez

Url: - https://github.com/Mordogon/03MIAR_Algoritmos-de-Optimizaci-n_Seminario -
<https://github.com/AgSanches/AlgoritmosOptimizaci-nTrabajo>

1.0.1 Problema 2. Organizar los horarios de partidos de La Liga

Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que **maximice la audiencia**.

Horarios disponibles Los horarios disponibles se conocen a priori y son los siguientes: - **Viernes:** 20:00 - **Sábado:** 12:00, 16:00, 18:00, 20:00 - **Domingo:** 12:00, 16:00, 18:00, 20:00 - **Lunes:** 20:00

Categorías y audiencia base En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores (que tiene relación directa con la audiencia). Hay **3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C**.

Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos):

	Categoría A	Categoría B	Categoría C
Categoría A	2 Millones	1.3 Millones	1 Millones
Categoría B		0.9 Millones	0.75 Millones
Categoría C			0.47 Millones

Ponderación por horario Si el horario del partido no se realiza a las 20 horas del sábado se sabe que la audiencia se reduce según los coeficientes de la siguiente tabla:

Horario	Viernes	Sábado	Domingo	Lunes
12h	-	0.55	0.45	-
16h	-	0.70	0.75	-
18h	-	0.80	0.85	-

Horario	Viernes	Sábado	Domingo	Lunes
20h	0.40	1.00	1.00	0.40

- **Restricción obligatoria:** debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes.

Penalización por coincidencias Es posible la coincidencia de horarios, pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Coincidencias	Penalización (-%)
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

....

(*) La respuesta es obligatoria

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

Posibilidades sin tener en cuenta las restricciones

Para resolver esta cuestión, primero se analizan los elementos de los que se dispone según el enunciado del problema: 1. Hay 10 partidos de fútbol que se deben programar en una jornada. 2. Existen 10 franjas horarias disponibles a lo largo del fin de semana (1 el viernes, 4 el sábado, 4 el domingo y 1 el lunes).

Dado que el enunciado permite que haya solapamientos (es decir, varios partidos pueden coincidir en la misma franja horaria sin invalidar la solución, aunque se aplique una penalización), se trata de un problema matemático de variaciones con repetición.

Para cada uno de los 10 partidos, se puede elegir cualquiera de los 10 horarios disponibles de forma independiente. Por tanto, el cálculo se formula de la siguiente manera:

$$\text{Variaciones con repetición} = m^n = 10^{10}$$

- Total de posibilidades sin restricciones: 10.000.000.000 (10 mil millones de combinaciones).

Posibilidades teniendo en cuenta todas las restricciones

El enunciado indica que existen únicamente dos restricciones obligatorias que invalidarían una solución si no se cumplen: 1. Se debe asignar, al menos, un partido al viernes a las 20h. 2. Se debe asignar, al menos, un partido al lunes a las 20h.

Para calcular matemáticamente las combinaciones válidas, es mucho más sencillo y eficiente utilizar el principio de inclusión-exclusión (calcular los casos totales y restar los casos que rompen las restricciones).

Se definen los siguientes escenarios inválidos: * Conjunto A (sin viernes): combinaciones donde no se juega ningún partido el viernes. Quedan 9 horarios disponibles para los 10 partidos, por lo que hay 9^{10} combinaciones inválidas. * Conjunto B (sin lunes): combinaciones donde no se juega ningún partido el lunes. Quedan 9 horarios disponibles para los 10 partidos, por lo que hay 9^{10} combinaciones inválidas. * Intersección A y B (ni viernes ni lunes): al restar los conjuntos A y B, se está restando dos veces el escenario donde no hay partidos ni el viernes ni el lunes. Se debe sumar esta intersección una vez para equilibrar matemáticamente la ecuación. Si se quitan ambos días, quedan 8 horarios para 10 partidos, resultando en 8^{10} combinaciones.

La fórmula de combinaciones válidas se expresa así:

$$Total\ vlidas = Total\ posible - A - B + (A \cap B)$$

$$Total\ vlidas = 10^{10} - 9^{10} - 9^{10} + 8^{10}$$

Se realiza el cálculo numérico: * $10^{10} = 10.000.000.000$ * $9^{10} = 3.486.784.401$ * $8^{10} = 1.073.741.824$

$$Total = 10.000.000.000 - 3.486.784.401 - 3.486.784.401 + 1.073.741.824$$

Total de posibilidades teniendo en cuenta las restricciones: 4.100.173.022 combinaciones válidas.

Modelo para el espacio de soluciones (*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Para abordar este problema computacionalmente y evaluar las soluciones de forma rápida, la estructura de datos idónea es una combinación de diccionarios y listas, debido a su eficiencia (búsquedas en tiempo constante) y claridad. Se proponen los siguientes elementos.

1. Audiencias base: un diccionario cuyas claves sean cadenas con las categorías de local y visitante ('Local_Visitante') y sus valores sean la audiencia base en millones. Como el orden local/visitante no afecta a la audiencia, se incluyen ambas combinaciones ('A_B' y 'B_A') para que la consulta sea directa. Por ejemplo: `audiencias_base = {'A_A': 2.0, 'A_B': 1.3, 'B_A': 1.3, 'C_C': 0.47, ...}`.
2. Ponderación de horarios: un diccionario donde la clave sea el identificador del horario y el valor sea el coeficiente reductor de audiencia. Por ejemplo: `modificadores_horario = {'Viernes_20': 0.4, 'Sabado_20': 1.0, ...}`.
3. Penalizaciones por coincidencia: un diccionario donde la clave represente el número de coincidencias y el valor sea el porcentaje final de audiencia tras aplicar la reducción. Por ejemplo: `penalizaciones = {0: 1.0, 1: 0.75, 2: 0.55, 3: 0.40, ...}`.

4. Espacio de soluciones: una lista de longitud 10 (una posición por cada partido a jugar) donde el valor almacenado sea la clave del horario asignado. Por ejemplo: `solucion = ['Viernes_20', 'Sabado_20', ...]`.

Esta estructura resulta muy útil porque separa por completo los datos estáticos de la variable dinámica. Esto permite programar la función de evaluación con un código limpio y evitar cálculos redundantes al acceder a los datos mediante una simple consulta directa.

```
[21]: # Estructura de datos para el problema:

horarios = [
    'Viernes_20',
    'Sabado_12', 'Sabado_16', 'Sabado_18', 'Sabado_20',
    'Domingo_12', 'Domingo_16', 'Domingo_18', 'Domingo_20',
    'Lunes_20'
]

partidos = [
    {'partido': 'Celta - Real Madrid', 'categorias': 'B_A'},
    {'partido': 'Valencia - R. Sociedad', 'categorias': 'B_A'},
    {'partido': 'Mallorca - Eibar', 'categorias': 'C_C'},
    {'partido': 'Athletic - Barcelona', 'categorias': 'B_A'},
    {'partido': 'Leganés - Osasuna', 'categorias': 'C_C'},
    {'partido': 'Villarreal - Granada', 'categorias': 'B_C'},
    {'partido': 'Alavés - Levante', 'categorias': 'B_B'},
    {'partido': 'Espanyol - Sevilla', 'categorias': 'B_B'},
    {'partido': 'Betis - Valladolid', 'categorias': 'B_C'},
    {'partido': 'Atlético - Getafe', 'categorias': 'B_B'}
]

audiencias_base = {
    'A_A': 2.0,
    'A_B': 1.3, 'B_A': 1.3,
    'A_C': 1.0, 'C_A': 1.0,
    'B_B': 0.9,
    'B_C': 0.75, 'C_B': 0.75,
    'C_C': 0.47
}

ponderaciones = {
    'Viernes_20': 0.4,
    'Sabado_12': 0.55,
    'Sabado_16': 0.7,
    'Sabado_18': 0.8,
    'Sabado_20': 1.0,
    'Domingo_12': 0.45,
    'Domingo_16': 0.75,
    'Domingo_18': 0.85,
```

```

'Domingo_20': 1.0,
'Lunes_20': 0.4
}

penalizaciones = {
    0: 1.0,    # 0% de reducción
    1: 0.75,   # 25% de reducción
    2: 0.55,   # 45% de reducción
    3: 0.40,   # 60% de reducción
    4: 0.30,   # 70% de reducción
    5: 0.25,   # 75% de reducción
    6: 0.22,   # 78% de reducción
    7: 0.20,   # 80% de reducción
    8: 0.20,   # 80% de reducción
    9: 0.20    # 80% de reducción
}

```

Según el modelo para el espacio de soluciones ***(*)** ¿Cual es la función objetivo?

***(*)** Según el modelo para el espacio de soluciones **(*)** ¿cuál es la función objetivo?

La función objetivo es la expresión matemática que calcula la audiencia total de una jornada en base a los horarios asignados al conjunto de partidos, aplicando los modificadores pertinentes. Se formula de la siguiente manera:

$$Audiencia\ Total = \sum_{i=1}^{10} (Base_i \times Ponderacion_Horario_i \times Penalizacion_Coincidencia_i)$$

En esta fórmula, las variables que intervienen se definen del siguiente modo: * $Base_i$: es la audiencia base del partido i según las categorías de los equipos que se enfrentan. * $Ponderacion_Horario_i$: es el coeficiente de reducción de audiencia que se aplica por jugar en un día y hora específicos. * $Penalizacion_Coincidencia_i$: es el porcentaje de audiencia restante tras aplicar la penalización correspondiente si existen otros partidos programados exactamente a la misma hora.

***(*)** ¿Es un problema de maximización o minimización?

Tal y como se establece en la descripción general del caso práctico, el propósito principal es organizar los horarios de los partidos de liga de cada jornada para conseguir la mayor cantidad de espectadores posibles. Por tanto, es un problema de maximización.

Diseña un algoritmo para resolver el problema por fuerza bruta

```

[22]: def evaluar_audiencia(solucion, partidos, audiencias_base, ponderaciones,
    ↪ penalizaciones):
    audiencia_total = 0

    # se recorren los 10 partidos programados en el calendario propuesto
    for i in range(len(solucion)):

```

```

    horario_asignado = solucion[i]

    # 1. obtener audiencia base según las categorías de los equipos
    base = audiencias_base[partidos[i]['categorias']]

    # 2. obtener modificador multiplicador por el día y hora jugados
    modificador = ponderaciones[horario_asignado]

    # 3. calcular penalización contando cuántos partidos coinciden a esa
    ↪misma hora
    # (se resta 1 para que el partido no se cuente a sí mismo como
    ↪coincidencia)
    coincidencias = solucion.count(horario_asignado) - 1
    penalizacion = penalizaciones[coincidencias]

    # se aplica la fórmula matemática de la función objetivo
    audiencia_partido = base * modificador * penalizacion

    # se acumula al total de la jornada
    audiencia_total += audiencia_partido

    return audiencia_total

def fuerza_bruta_liga(partidos, horarios, audiencias_base, ponderaciones,
    ↪penalizaciones):
    mejor_audiencia = 0
    mejor_solucion = None

    # generación exhaustiva mediante bucles anidados para los 10 partidos
    for p1 in horarios:
        for p2 in horarios:
            for p3 in horarios:
                for p4 in horarios:
                    for p5 in horarios:
                        for p6 in horarios:
                            for p7 in horarios:
                                for p8 in horarios:
                                    for p9 in horarios:
                                        for p10 in horarios:

                                            # se agrupan los horarios asignados
                                            ↪en una lista
                                            solucion = [p1, p2, p3, p4, p5, p6,
                                            ↪p7, p8, p9, p10]

                                            # comprobar restricciones
                                            ↪obligatorias

```

```

if 'Viernes_20' not in solucion or
↳'Lunes_20' not in solucion:
    continue

    # evaluar la función objetivo
    audiencia_actual =
↳evaluar_audiencia(solucion, partidos, audiencias_base, ponderaciones,
↳penalizaciones)

    # actualizar si es la mejor hasta
↳ahora

    if audiencia_actual >
↳mejor_audiencia:
        mejor_audiencia =
↳audiencia_actual

        mejor_solucion = solucion

return mejor_solucion, mejor_audiencia

```

Aunque nos gustaría mostrar la ejecución del algoritmo, debido al gran coste computacional, resulta muy complicado ejecutarlo y mostrar sus resultados porque podría tardar una gran cantidad de tiempo en ejecutarse. Por lo tanto, solo se muestra como referencia para comparar con otros algoritmos en apartados posteriores.

Con el fin de estimar el tiempo necesario para ejecutar el algoritmo de fuerza bruta, se ha elaborado el siguiente código. A partir del número total de soluciones posibles y del número de soluciones que cumplen las restricciones especificadas, se mide el coste medio de generar y comprobar una solución, así como el coste medio de evaluar una solución válida. Estos valores permiten conocer el tiempo necesario para recorrer todo el espacio de soluciones.

Como se puede observar, la ejecución completa se estima en unas 2 horas en la máquina utilizada para realizar las pruebas (y el tiempo se multiplica por 10 con cada partido adicional), por lo que ejecutar el algoritmo de fuerza bruta de forma exhaustiva supone un coste computacional elevado.

```

[23]: import time
import random

# Medimos el coste real de una iteración de la fuerza bruta y extrapolamos al
↳total.
# Muestra de soluciones aleatorias (la proporción de válidas es similar a la
↳real, ~41 %).
random.seed(0)
muestra = 100_000
soluciones_muestra = [[random.choice(horarios) for _ in range(10)] for _ in
↳range(muestra)]

# (a) coste de construir la solución y comprobar las restricciones (se hace en
↳las 10^10 iteraciones)

```

```

t0 = time.perf_counter()
for s in soluciones_muestra:
    solucion = list(s)
    if 'Viernes_20' not in solucion or 'Lunes_20' not in solucion:
        continue
t_gen = (time.perf_counter() - t0) / muestra

# (b) coste de evaluar_audiencia (solo en las soluciones válidas)
t0 = time.perf_counter()
for s in soluciones_muestra:
    evaluar_audiencia(s, partidos, audiencias_base, ponderaciones,
        ↪penalizaciones)
t_eval = (time.perf_counter() - t0) / muestra

total_iter = 10**10
validas = 10**10 - 2 * 9**10 + 8**10
horas = (t_gen * total_iter + t_eval * validas) / 3600
print(f"Coste medio por iteración (construir + comprobar): {t_gen * 1e6:.2f}
    ↪microsegundos")
print(f"Coste medio de evaluar_audiencia: {t_eval * 1e6:.2f}
    ↪microsegundos")
print(f"Soluciones generadas: {total_iter:,} | válidas (evaluadas): {validas:
    ↪,} ".replace(",", "."))
print(f"Tiempo total estimado en esta máquina: {horas:.1f} horas")
print(f"Con un partido más (11 partidos y 10 franjas) sería {horas * 10:.0f}
    ↪horas")

```

Coste medio por iteración (construir + comprobar): 0.12 microsegundos
 Coste medio de evaluar_audiencia: 1.41 microsegundos
 Soluciones generadas: 10.000.000.000 | válidas (evaluadas): 4.100.173.022
 Tiempo total estimado en esta máquina: 1.9 horas
 Con un partido más (11 partidos y 10 franjas) sería 19 horas

Calcula la complejidad del algoritmo por fuerza bruta

Complejidad Fuerza bruta Notación: n = número de partidos (10) y m = número de franjas horarias (10).

1) Número de asignaciones generadas. Los 10 bucles anidados generan todas las variaciones con repetición: $m^n = 10^{10}$. El filtro de restricciones ('Viernes_20' not in solucion ...) se aplica *después* de generar cada asignación, así que se recorren las 10^{10} , de las cuales solo 4.100.173.022 son válidas y llegan a evaluarse.

2) Coste por asignación. Hay que tener en cuenta lo que hace realmente el código:

- Construir la lista `solucion` y comprobar las dos restricciones con `in`: $O(n)$.
- `evaluar_audiencia` (solo para las válidas) tiene un bucle de n iteraciones y **en cada una llama a `solucion.count(horario_asignado)`, que recorre toda la lista**, es decir $O(n)$. El coste de la evaluación es por tanto $O(n \cdot n) = O(n^2)$ y **no** $O(n)$.

3) Complejidad total.

$$O(n^2 \cdot m^n)$$

Con $n = m = 10$ son del orden de $10^2 \cdot 10^{10} = 10^{12}$ operaciones elementales (un billón).

Espacio: $O(n)$ (solo se guarda la asignación actual y la mejor encontrada).

La celda anterior mide el coste real de una iteración y extrapola el tiempo total: del orden de horas en un ordenador moderno (y bastante más en un entorno como Colab, con sesiones limitadas), y se multiplica por 10 con cada partido adicional. Por eso se considera impracticable y se muestra solo como referencia.

(*) Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Mejora del algoritmo (1): heurística voraz (*Greedy*) La idea consiste en ordenar los partidos de la jornada por audiencia base (según la categoría de los equipos) y, por otra parte, las franjas por su coeficiente de audiencia. Después se asigna al horario con mayor coeficiente el partido con mayor audiencia esperada, y así sucesivamente hasta rellenar la jornada. Cada franja recibe como máximo un partido, de modo que no se producen coincidencias ($Penalizacion_Coincidencia_i = 1$).

La función objetivo es la misma de antes:

$$Audiencia\ Total = \sum_{i=1}^{10} (Base_i \times Ponderacion_Horario_i \times Penalizacion_Coincidencia_i)$$

Qué se puede afirmar y qué no sobre su optimalidad. - Entre las asignaciones *sin coincidencias* (permutaciones de partidos en franjas), emparejar los mayores con los mayores es óptimo por la desigualdad del reordenamiento. - Sin embargo, **no se puede garantizar que el óptimo global del problema carezca de coincidencias**. El enunciado solo obliga a ocupar el viernes y el lunes; el resto de franjas pueden quedar vacías. Por ello puede compensar dejar vacía una franja de coeficiente bajo (por ejemplo, domingo 12h, 0,45) y juntar dos partidos en una franja de coeficiente alto aun pagando el 25 % de penalización. Más abajo se muestra un contraejemplo concreto y el estudio empírico cuantifica con qué frecuencia ocurre.

Por tanto, el algoritmo voraz es una **heurística**: muy rápida, $O(n \log n)$, pero sin garantía de óptimo.

```
[24]: def greedy_audiencia_base_por_partido(partidos, audiencias_base):
    audiencia_por_partido = []

    for idx, partido in enumerate(partidos):
        categoria_partido = partido["categorias"]
        audiencia_por_partido.append({"idx": idx, "audiencia_base":
        ↪audiencias_base[categoria_partido]})

    return sorted(
```

```

    audiencia_por_partido,
    key = lambda x: x["audiencia_base"],
    reverse=True
)

def evaluar_audiencia_greedy(solucion, partidos, audiencias_base,
⇨ponderaciones):
    audiencia_total = 0

    for idx, horario in enumerate(solucion):
        audiencia_base = audiencias_base[partidos[idx]["categorias"]]
        audiencia_total += audiencia_base * ponderaciones[horario]

    return audiencia_total

def greedy_liga(partidos, audiencias_base, ponderaciones):
    # Se ordenan los partidos por audiencia base.
    partidos_ordenados_audiencia = greedy_audiencia_base_por_partido(partidos,
⇨audiencias_base)

    # Para cada horario, se observa sus coeficientes de audiencia
    horarios_ordenados = [
        horario
        for horario, _ in sorted(
            ponderaciones.items(),
            key=lambda x: x[1],
            reverse=True
        )
    ]
    solucion = [None] * len(partidos_ordenados_audiencia)

    # Se asigna al mejor partido al mejor horario disponible.
    for partido, horario in zip(partidos_ordenados_audiencia, horarios_ordenados):
        solucion[partido["idx"]] = horario

    audiencia = evaluar_audiencia_greedy(
        solucion,
        partidos,
        audiencias_base,
        ponderaciones
    )

    return solucion, audiencia

solucion, audiencia_jornada = greedy_liga(partidos, audiencias_base,
⇨ponderaciones)

```

```

for i, horario in enumerate(solucion):
    print(f"{partidos[i]['partido']} -> {horario}")

print(f"\nAudiencia total para la jornada: {audiencia_jornada:.2f} Mill.")

```

```

Celta - Real Madrid -> Sabado_20
Valencia - R. Sociedad -> Domingo_20
Mallorca - Eibar -> Viernes_20
Athletic - Barcelona -> Domingo_18
Leganés - Osasuna -> Lunes_20
Villarreal - Granada -> Sabado_12
Alavés - Levante -> Sabado_18
Espanyol - Sevilla -> Domingo_16
Betis - Valladolid -> Domingo_12
Atlético - Getafe -> Sabado_16

```

Audiencia total para la jornada: 6.86 Mill.

Mejora del algoritmo (2): programación dinámica sobre subconjuntos (óptimo exacto)

Como el voraz no garantiza el óptimo, se propone además un algoritmo **exacto** que no es voraz y que mejora de forma drástica la complejidad de la fuerza bruta.

Observación clave. La audiencia de una franja solo depende del *conjunto* de partidos que se asignan a ella: si el conjunto T tiene k partidos, la audiencia de esa franja es

$$valor(s, T) = \left(\sum_{i \in T} Base_i \right) \times Coef_s \times Penalizacion[k - 1]$$

Como la función objetivo es la suma de los valores de las franjas, el problema se puede resolver franja a franja.

Subproblema. $f(s, S)$ = máxima audiencia posible al repartir el subconjunto S de partidos entre las s primeras franjas.

Recurrencia.

$$f(s, S) = \max_{T \subseteq S} \left[f(s - 1, S \setminus T) + valor(s, T) \right]$$

con $f(0, \emptyset) = 0$, $f(0, S \neq \emptyset) = -\infty$, y la restricción de que $T \neq \emptyset$ cuando la franja s es el viernes o el lunes. La solución es $f(m, \text{todos})$, y guardando qué T se eligió en cada paso se reconstruye la asignación.

Como T puede ser vacío, el algoritmo considera de forma natural dejar franjas sin usar y colocar varios partidos en una misma franja, es decir, **explora los solapamientos** que el voraz ignora.

```

[25]: def dp_liga(partidos, horarios, audiencias_base, ponderaciones, penalizaciones):
        # Programación dinámica sobre subconjuntos: devuelve (solucion, audiencia)
        ↪ óptimos.
        n, m = len(partidos), len(horarios)

```

```

bases = [audiencias_base[p["categorias"]] for p in partidos]
coef = [ponderaciones[h] for h in horarios]
obligatorias = {horarios.index("Viernes_20"), horarios.index("Lunes_20")}
TODOs = (1 << n) - 1

# Precálculo para cada subconjunto (máscara de bits): suma de audiencias
↳ base y n° de partidos
suma = [0.0] * (1 << n)
tam = [0] * (1 << n)
for S in range(1, 1 << n):
    bajo = S & -S
    suma[S] = suma[S ^ bajo] + bases[bajo.bit_length() - 1]
    tam[S] = tam[S ^ bajo] + 1

NO_ALCANZABLE = float("-inf")
f = [NO_ALCANZABLE] * (1 << n)
f[0] = 0.0
previo = [] # previo[s][S] = partidos ya colocados antes de la franja s
↳ (para reconstruir)

for s in range(m):
    g = [NO_ALCANZABLE] * (1 << n)
    prev = [0] * (1 << n)
    for S in range(1 << n): # S = partidos ya asignados a
↳ las franjas anteriores
        if f[S] == NO_ALCANZABLE:
            continue
        libres = TODOs ^ S
        T = libres # T = partidos que se asignan a
↳ la franja s
        while True: # se recorren todos los
↳ subconjuntos T de 'libres'
            if T or s not in obligatorias: # viernes y lunes no pueden
↳ quedar vacíos
                valor = f[S] + (suma[T] * coef[s] * penalizaciones[tam[T] -
↳ 1] if T else 0.0)
                if valor > g[S | T]:
                    g[S | T] = valor
                    prev[S | T] = S
                if T == 0:
                    break
                T = (T - 1) & libres
        f = g
        previo.append(prev)

# Reconstrucción de la solución recorriendo las franjas hacia atrás

```

```

solucion = [None] * n
S = TODOS
for s in range(m - 1, -1, -1):
    S_prev = previo[s][S]
    T = S ^ S_prev
    for i in range(n):
        if (T >> i) & 1:
            solucion[i] = horarios[s]
    S = S_prev

return solucion, f[TODOS]

# 1) Resultado sobre la jornada del enunciado
solucion_dp, audiencia_dp = dp_liga(partidos, horarios, audiencias_base,
    ⇨ponderaciones, penalizaciones)
for i, horario in enumerate(solucion_dp):
    print(f"{partidos[i]['partido']} -> {horario}")
print(f"\nAudiencia total (óptimo exacto): {audiencia_dp:.3f} Mill.")
print(f"Comprobación con la función objetivo original: "
    f"{evaluar_audiencia(solucion_dp, partidos, audiencias_base,
    ⇨ponderaciones, penalizaciones):.3f} Mill.")

```

```

Celta - Real Madrid -> Sabado_20
Valencia - R. Sociedad -> Domingo_18
Mallorca - Eibar -> Viernes_20
Athletic - Barcelona -> Domingo_20
Leganés - Osasuna -> Lunes_20
Villarreal - Granada -> Sabado_12
Alavés - Levante -> Sabado_16
Espanyol - Sevilla -> Sabado_18
Betis - Valladolid -> Domingo_12
Atlético - Getafe -> Domingo_16

```

```

Audiencia total (óptimo exacto): 6.856 Mill.
Comprobación con la función objetivo original: 6.856 Mill.

```

Validación de la programación dinámica frente a la fuerza bruta. Como la fuerza bruta completa es inviable, se comprueba que ambos algoritmos coinciden en instancias reducidas (6 partidos y 6 franjas, incluyendo viernes y lunes), donde la fuerza bruta sí termina ($6^6 = 46.656$ asignaciones).

```

[26]: import itertools
import random

random.seed(11)
horarios_red = ['Viernes_20', 'Sabado_16', 'Sabado_20', 'Domingo_18',
    ⇨'Domingo_20', 'Lunes_20']

```

```

categorias_posibles = list(audiencias_base.keys())

for prueba in range(5):
    partidos_red = [{"partido": f"P{i+1}", "categorias": random.
    ↪choice(categorias_posibles)} for i in range(6)]

    # Fuerza bruta reducida (mismo criterio de validez: viernes y lunes
    ↪ocupados)
    mejor_fb = 0
    for sol in itertools.product(horarios_red, repeat=6):
        if 'Viernes_20' in sol and 'Lunes_20' in sol:
            mejor_fb = max(mejor_fb, evaluar_audiencia(list(sol), partidos_red,
            ↪audiencias_base, ponderaciones, penalizaciones))

    _, mejor_dp = dp_liga(partidos_red, horarios_red, audiencias_base,
    ↪ponderaciones, penalizaciones)
    print(f"Prueba {prueba+1}: fuerza bruta = {mejor_fb:.4f} | prog. dinámica =
    ↪{mejor_dp:.4f} | coinciden: {abs(mejor_fb - mejor_dp) < 1e-9}")

```

Prueba 1: fuerza bruta = 3.2885 | prog. dinámica = 3.2885 | coinciden: True
 Prueba 2: fuerza bruta = 4.7180 | prog. dinámica = 4.7180 | coinciden: True
 Prueba 3: fuerza bruta = 5.5930 | prog. dinámica = 5.5930 | coinciden: True
 Prueba 4: fuerza bruta = 6.5030 | prog. dinámica = 6.5030 | coinciden: True
 Prueba 5: fuerza bruta = 6.2100 | prog. dinámica = 6.2100 | coinciden: True

Contraejemplo: el óptimo global puede tener coincidencias. Jornada válida con 3 partidos A-C (1,0 M), 4 partidos B-B (0,9 M) y 3 partidos B-C (0,75 M); es decir, 3 equipos de categoría A, 11 de B y 6 de C, como en el enunciado.

```

[27]: from collections import Counter

cats_contraejemplo = ["A_C"] * 3 + ["B_B"] * 4 + ["B_C"] * 3
contraejemplo = [{"partido": f"P{i+1}", "categorias": c} for i, c in
    ↪enumerate(cats_contraejemplo)]

sol_voraz, aud_voraz = greedy_liga(contraejemplo, audiencias_base,
    ↪ponderaciones)
sol_dp, aud_dp = dp_liga(contraejemplo, horarios, audiencias_base,
    ↪ponderaciones, penalizaciones)

print(f"Voraz (sin coincidencias): {aud_voraz:.4f} Mill.")
print(f"Programación dinámica:      {aud_dp:.4f} Mill. (mejora de {aud_dp -
    ↪aud_voraz:.4f} Mill.)")
print("\nOcupación de franjas en la solución óptima:")
ocupacion = Counter(sol_dp)
for h in horarios:
    print(f"  {h:<11} -> {ocupacion.get(h, 0)} partido(s)")

```

Voraz (sin coincidencias): 6.3075 Mill.

Programación dinámica: 6.3175 Mill. (mejora de 0.0100 Mill.)

Ocupación de franjas en la solución óptima:

```
Viernes_20 -> 1 partido(s)
Sabado_12  -> 1 partido(s)
Sabado_16  -> 1 partido(s)
Sabado_18  -> 1 partido(s)
Sabado_20  -> 1 partido(s)
Domingo_12 -> 0 partido(s)
Domingo_16 -> 1 partido(s)
Domingo_18 -> 1 partido(s)
Domingo_20 -> 2 partido(s)
Lunes_20   -> 1 partido(s)
```

Mejora del algoritmo (3): recocido simulado (metaheurística) La programación dinámica nos da el óptimo exacto, pero su coste $O(m \cdot 3^n)$ sigue creciendo de forma exponencial con el número de partidos. Por eso buscamos también un método que pueda escalar a jornadas más grandes y que, a diferencia del voraz, **sí se atreva a probar coincidencias**. Para ello implementamos un **recocido simulado**, que funciona así:

1. **Punto de partida:** no empezamos de cero, sino desde la solución que nos da el algoritmo voraz. Así el recocido arranca desde una solución que ya es buena y solo tiene que mejorarla.
2. **Cómo se mueve:** en cada paso hacemos un pequeño cambio al azar sobre la solución actual. La mitad de las veces cogemos un partido y lo cambiamos a otra franja cualquiera. Así pueden aparecer o desaparecer coincidencias, que es justo lo que el voraz nunca prueba. La otra mitad, intercambiamos los horarios de dos partidos. Si el cambio deja el viernes o el lunes sin partido, lo descartamos directamente.
3. **Cuándo aceptamos el cambio:** si el cambio mejora la audiencia, nos lo quedamos siempre. Si la empeora, a veces también lo aceptamos, con una probabilidad $e^{\Delta/T}$ que es más alta cuanto menor es la pérdida. Puede parecer raro, pero aceptar a veces una solución peor permite llegar más adelante a soluciones mejores.
4. **Enfriamiento:** la temperatura T marca cuánto “se arriesga” el algoritmo. Empieza alta ($T_0 = 0,2$), y al principio acepta bastantes empeoramientos y explora mucho. En cada paso la multiplicamos por $\alpha = 0,999$, así que va bajando poco a poco. Tras las $K = 5.000$ iteraciones está tan baja que prácticamente solo acepta mejoras y se dedica a afinar la solución.

```
[28]: import math
from collections import Counter

def evaluar_rapido(solucion, bases, ponderaciones, penalizaciones):
    # Misma función objetivo que evaluar_audiencia, pero en O(n): las
    ↪coincidencias se cuentan una sola vez
    ocupacion = Counter(solucion)
    return sum(b * ponderaciones[h] * penalizaciones[ocupacion[h] - 1] for b, h
    ↪in zip(bases, solucion))
```

```

def recocido_simulado_liga(partidos, horarios, audiencias_base, ponderaciones,
    penalizaciones,
    t_inicial=0.2, enfriamiento=0.999, iteraciones=5000,
    semilla=None):
    rng = random.Random(semilla)
    n = len(partidos)
    bases = [audiencias_base[p["categorias"]] for p in partidos]
    obligatorias = ("Viernes_20", "Lunes_20")

    # Solución inicial: la del algoritmo voraz
    actual, _ = greedy_liga(partidos, audiencias_base, ponderaciones)
    valor_actual = evaluar_rapido(actual, bases, ponderaciones, penalizaciones)
    mejor, valor_mejor = actual[:], valor_actual
    t = t_inicial

    for _ in range(iteraciones):
        # Generación de un vecino: mover un partido de franja o intercambiar
        dos partidos
        vecino = actual[:]
        if rng.random() < 0.5:
            vecino[rng.randrange(n)] = rng.choice(horarios)
        else:
            i, j = rng.sample(range(n), 2)
            vecino[i], vecino[j] = vecino[j], vecino[i]

        # Restricción obligatoria: viernes y lunes ocupados
        if any(h not in vecino for h in obligatorias):
            t *= enfriamiento
            continue

        valor_vecino = evaluar_rapido(vecino, bases, ponderaciones,
            penalizaciones)
        delta = valor_vecino - valor_actual

        # Criterio de Metrópolis: se aceptan siempre las mejoras y a veces los
        empeoramientos
        if delta >= 0 or rng.random() < math.exp(delta / t):
            actual, valor_actual = vecino, valor_vecino
            if valor_actual > valor_mejor:
                mejor, valor_mejor = actual[:], valor_actual

        t *= enfriamiento

    return mejor, valor_mejor

```



```

# 1) Jornada del enunciado
sol_rs, aud_rs = recocido_simulado_liga(partidos, horarios, audiencias_base,
    ⇨ponderaciones, penalizaciones, semilla=0)
for i, horario in enumerate(sol_rs):
    print(f"{partidos[i]['partido']} -> {horario}")
print(f"\nRecocido simulado: {aud_rs:.4f} Mill. | óptimo (PD): {audiencia_dp:.4f} Mill.")

# 2) Contraejemplo en el que el voraz no es óptimo
sol_rs_ce, aud_rs_ce = recocido_simulado_liga(contraejemplo, horarios,
    ⇨audiencias_base, ponderaciones, penalizaciones, semilla=0)
print(f"\nContraejemplo -> voraz: {aud_voraz:.4f} | recocido simulado:
    ⇨{aud_rs_ce:.4f} | óptimo (PD): {aud_dp:.4f} Mill.")
print("Franjas con coincidencias en la solución del recocido:",
    {h: c for h, c in Counter(sol_rs_ce).items() if c > 1})

```

Celta - Real Madrid -> Sabado_20
 Valencia - R. Sociedad -> Domingo_20
 Mallorca - Eibar -> Viernes_20
 Athletic - Barcelona -> Domingo_18
 Leganés - Osasuna -> Lunes_20
 Villarreal - Granada -> Sabado_12
 Alavés - Levante -> Sabado_18
 Espanyol - Sevilla -> Domingo_16
 Betis - Valladolid -> Domingo_12
 Atlético - Getafe -> Sabado_16

Recocido simulado: 6.8560 Mill. | óptimo (PD): 6.8560 Mill.

Contraejemplo -> voraz: 6.3075 | recocido simulado: 6.3175 | óptimo (PD): 6.3175 Mill.

Franjas con coincidencias en la solución del recocido: {'Domingo_20': 2}

(*)Calcula la complejidad del algoritmo

1.1 Complejidad

Notación: n = número de partidos y m = número de franjas (en este problema $n = m = 10$).

Algoritmo voraz (heurística). - Ordenar los partidos por audiencia base: $O(n \log n)$. - Ordenar las franjas por coeficiente: $O(m \log m)$. - Asignar recorriendo ambas listas y evaluar la solución: $O(n)$.

La complejidad total es $O(n \log n + m \log m)$, que para $n = m$ es $O(n \log n)$. Es la más rápida, pero **no garantiza el óptimo** (ver contraejemplo y estudio empírico).

Programación dinámica sobre subconjuntos (exacta). - Precálculo de la suma de audiencias y del tamaño de cada subconjunto de partidos: $O(2^n)$. - Para cada franja (m) se recorren todos los pares (S, T) con $T \subseteq S$ (partidos no incluidos en S). El número de pares es $\sum_S 2^{n-|S|} = 3^n$ y cada transición cuesta $O(1)$ gracias al precálculo. - Reconstrucción de la solución: $O(m \cdot n)$.

$$T(n, m) = O(m \cdot 3^n) \quad \text{Memoria: } O(m \cdot 2^n)$$

(la memoria corresponde a los punteros que se guardan para reconstruir la asignación; si solo se quisiera el valor óptimo bastaría $O(2^n)$).

Recocido simulado (metaheurística). - Solución inicial con el voraz: $O(n \log n)$. - En cada una de las K iteraciones se genera un vecino (copia de la solución, $O(n)$), se comprueban las franjas obligatorias ($O(n)$) y se evalúa con `evaluar_rapido`, que cuenta las coincidencias en una sola pasada con `Counter`: $O(n)$.

$$T(n, K) = O(n \log n + K \cdot n) = O(K \cdot n) \quad \text{Memoria: } O(n + m)$$

Con $K = 5.000$ y $n = 10$ son unas $5 \cdot 10^4$ operaciones. El coste ya no es exponencial en n , sino que lo fija el número de iteraciones, por lo que escala a jornadas mucho mayores. A cambio, no garantiza el óptimo.

Comparación con la fuerza bruta ($O(n^2 \cdot m^n)$):

Algoritmo	Complejidad	Operaciones con $n = m = 10$	¿Óptimo?
Fuerza bruta	$O(n^2 m^n)$	$\approx 10^{12}$	Sí
Voraz	$O(n \log n)$	$\approx 10^2$	No garantizado
Prog. dinámica	$O(m \cdot 3^n)$	$\approx 5,9 \cdot 10^5$	Sí
Recocido simulado	$O(K \cdot n)$	$\approx 5 \cdot 10^4$	No garantizado

La programación dinámica reduce el coste en un factor del orden de 10^6 manteniendo la garantía de optimalidad. Sigue siendo exponencial en n , pero el número de partidos por jornada es pequeño y fijo (10), por lo que es perfectamente viable aquí; para valores de n mayores ($3^{21} \approx 10^{10}$) dejaría de serlo y habría que recurrir a metaheurísticas como el recocido simulado.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Datos de entrada

En primer lugar, vamos a generar una clase para definir los “Equipos” con tres atributos: - ID - Nombre - Categoría

Para que a continuación nos resulte más sencillo generar la jornada en cuestión. Y se genera una lista con todos los equipos y sus categorías correspondientes.

Respuesta

```
[29]: from dataclasses import dataclass
import random

@dataclass(frozen=True)
class Equipo:
    id: int
    nombre: str
```

```

    categoria: str

equipos = [
    Equipo(1, "Real Madrid", "A"),
    Equipo(2, "Barcelona", "A"),
    Equipo(3, "Real Sociedad", "A"),

    Equipo(4, "Atlético", "B"),
    Equipo(5, "Athletic", "B"),
    Equipo(6, "Valencia", "B"),
    Equipo(7, "Villarreal", "B"),
    Equipo(8, "Alavés", "B"),
    Equipo(9, "Espanyol", "B"),
    Equipo(10, "Betis", "B"),
    Equipo(11, "Celta", "B"),
    Equipo(12, "Levante", "B"),
    Equipo(13, "Getafe", "B"),
    Equipo(14, "Sevilla", "B"),

    Equipo(15, "Mallorca", "C"),
    Equipo(16, "Eibar", "C"),
    Equipo(17, "Leganés", "C"),
    Equipo(18, "Osasuna", "C"),
    Equipo(19, "Granada", "C"),
    Equipo(20, "Valladolid", "C"),
]

```

1.1.1 Generar la jornada

Para generar jornadas de manera aleatoria implementamos el método `generar_jornada`, el cual a partir de un conjunto de equipos genera una jornada con los equipos de una manera totalmente aleatoria.

Con ello podemos probar el algoritmo con jornadas aleatorias.

```

[30]: def generar_jornada(equipos):
    partidos = []
    copy_equipos = equipos[:]

    random.shuffle(copy_equipos)

    for i in range(0, len(copy_equipos), 2):
        local = copy_equipos[i]
        visitante = copy_equipos[i + 1]

        partidos.append({
            "partido": f"{local.nombre} - {visitante.nombre}",
            "categorias": f"{local.categoria}_{visitante.categoria}"

```

```

    })

    return partidos

random.seed(42)
jornada = generar_jornada(equipos)
solucion, audiencia_jornada = greedy_liga(jornada, audiencias_base,
    ponderaciones)

for i, horario in enumerate(solucion):
    print(f"{jornada[i]['partido']} -> {horario}")

print(f"\nAudiencia total para la jornada: {audiencia_jornada:.2f} Mill.")

```

```

Valladolid - Valencia -> Sabado_12
Mallorca - Athletic -> Domingo_12
Betis - Sevilla -> Sabado_18
Eibar - Granada -> Lunes_20
Villarreal - Getafe -> Domingo_16
Osasuna - Celta -> Viernes_20
Barcelona - Levante -> Sabado_20
Real Sociedad - Leganés -> Domingo_18
Alavés - Espanyol -> Sabado_16
Real Madrid - Atlético -> Domingo_20

```

Audiencia total para la jornada: 6.71 Mill.

1.1.2 Generar calendario completo

El calendario de la liga de fútbol consta de 38 jornadas si contamos con 20 equipos, donde cada equipo juega de local y visitante ante cada equipo. Desde el año 2018 se cuenta con un calendario totalmente asimétrico, es decir los partidos de la primera y segunda vuelta no se juegan en el mismo orden, es decir no hay relación entre ambas vueltas.

Pero para generar un calendario asimétrico se necesita un algoritmo bastante complejo de implementar con algunas restricciones, por lo tanto se propone adoptar el algoritmo simétrico para generar las jornadas de una manera sencilla y evaluar únicamente los algoritmos de optimización implementados anteriormente.

Para implementar el calendario simétrico, se necesita de un algoritmo “circular” conocido como “Round robin” [2], donde cada equipo obtiene un orden y se van emparejando los equipos según las posiciones y en cada jornada se cambia el orden de una manera circular y se vuelven a emparejar los equipos.

Únicamente hacen falta generar las primeras 19 jornadas y las siguientes 19 serán iguales pero invirtiendo el orden de local y visitante.

Posteriormente utilizamos el algoritmo de optimización para asignar horarios, observar la audiencia potencial por jornada y al final se hace el recuento de la audiencia de la liga al completo.

```

[31]: # Round Robin

def generar_jornada_robin(equipos_jornada, invertir_local_visitante = False):
    partidos = []
    partidos_jornada = len(equipos_jornada) // 2

    for i in range(partidos_jornada):
        local = i
        visitante = -(i + 1)

        if invertir_local_visitante:
            local, visitante = visitante, local

        local = equipos_jornada[local]
        visitante = equipos_jornada[visitante]

        partidos.append({
            "partido": f"{local.nombre} - {visitante.nombre}",
            "categorias": f"{local.categoria}_{visitante.categoria}"
        })

    return partidos

def generar_calendario_robin(equipos_calendario):
    # Calcular cantidad de jornadas ida.
    cantidad_equipos = len(equipos_calendario)
    cantidad_de_jornadas = cantidad_equipos - 1
    calendario = [None] * (cantidad_de_jornadas * 2)

    equipos_round_robin = equipos_calendario[:]
    random.shuffle(equipos_round_robin)

    for i in range(cantidad_de_jornadas):
        calendario[i] = generar_jornada_robin(equipos_round_robin)
        calendario[cantidad_de_jornadas + i] = □
        ↪ generar_jornada_robin(equipos_round_robin, invertir_local_visitante=True)

        robin_fijo = equipos_round_robin[0:1]
        robin_variable = equipos_round_robin[-1:] + equipos_round_robin[1:-1]
        equipos_round_robin = robin_fijo + robin_variable

    return calendario

# Generamos el calendario
random.seed(42)
calendario_partido = generar_calendario_robin(equipos)
suma_audiencia_total = 0

```

```

for idx_jornada, jornada in enumerate(calendario_partido):
    solucion, audiencia_jornada = greedy_liga(jornada, audiencias_base,
    ↪ponderaciones)
    suma_audiencia_total += audiencia_jornada

    # Se muestra en detalle solo la primera jornada para no alargar la salida
    if idx_jornada == 0:
        print(f"{'=' * 20} Jornada {idx_jornada + 1} {'=' * 20}")
        for i, horario in enumerate(solucion):
            print(f"{jornada[i]['partido']:<35} → {horario}")
        print("-" * 60)
        print(f"Audiencia de la jornada: {audiencia_jornada:.2f} millones de
        ↪espectadores.")

print("\n" + "=" * 60)
print(f"Audiencia total del calendario: {suma_audiencia_total:.2f} millones de
↪espectadores")
print("=" * 60)

```

===== Jornada 1 =====

Valladolid - Atlético	→ Sabado_16
Valencia - Real Madrid	→ Sabado_20
Mallorca - Espanyol	→ Sabado_12
Athletic - Alavés	→ Sabado_18
Betis - Leganés	→ Domingo_12
Sevilla - Real Sociedad	→ Domingo_20
Eibar - Levante	→ Viernes_20
Granada - Barcelona	→ Domingo_18
Villarreal - Celta	→ Domingo_16
Getafe - Osasuna	→ Lunes_20

Audiencia de la jornada: 6.72 millones de espectadores.

=====

Audiencia total del calendario: 256.90 millones de espectadores

=====

1.1.3 Estudio empírico de las heurísticas (voraz y recocido simulado)

Se pide evaluar cómo de lejos se quedan las heurísticas del óptimo y qué variabilidad tiene esa diferencia. Como disponemos de un algoritmo exacto (programación dinámica, validado contra la fuerza bruta), el **óptimo real** de cada jornada se calcula sin necesidad de simularlo.

Para cada jornada aleatoria se calcula: - Diferencia absoluta: $\Delta = \text{Optimo} - \text{Heuristica}$ (millones de espectadores), donde *Heuristica* es la audiencia obtenida por el voraz o por el recocido simulado.
- Diferencia relativa: $100 \cdot \Delta / \text{Optimo}$ (%).

Como la audiencia solo depende de las audiencias base de los 10 partidos (no de los nombres),

el óptimo se guarda en una caché indexada por ese multiconjunto de audiencias, lo que permite evaluar muchas jornadas sin repetir cálculos.

```
[32]: import statistics

def perfil(jornada):
    # Clave de caché: multiconjunto de audiencias base de la jornada
    return tuple(sorted(audiencias_base[p["categorias"]] for p in jornada))

def resumen(nombre, valores):
    print(f"{nombre}: media = {statistics.mean(valores):.5f} | desv. típica = {statistics.pstdev(valores):.5f} "
          f"| varianza = {statistics.pvariance(valores):.2e} | máximo = {max(valores):.5f}")

# --- Experimento 1: jornadas aleatorias de la liga (20 equipos, 10 partidos) ---
random.seed(2024)
N = 10_000
cache_optimo = {}
dif_abs, dif_rel = [], []

for _ in range(N):
    jornada = generar_jornada(equipos)
    clave = perfil(jornada)
    if clave not in cache_optimo:
        cache_optimo[clave] = dp_liga(jornada, horarios, audiencias_base, ponderaciones, penalizaciones)[1]
    optimo = cache_optimo[clave]
    _, voraz = greedy_liga(jornada, audiencias_base, ponderaciones)
    dif_abs.append(optimo - voraz)
    dif_rel.append(100 * (optimo - voraz) / optimo)

subopt = [d for d in dif_abs if d > 1e-9]
print(f"Experimento 1: {N} jornadas aleatorias ({len(cache_optimo)} perfiles de audiencia distintos)")
print(f"Jornadas en las que el voraz alcanza el óptimo: {100 * (N - len(subopt)) / N:.2f} %")
resumen("Diferencia absoluta (Mill.)", dif_abs)
resumen("Diferencia relativa (%)", dif_rel)
if subopt:
    print(f"Cuando el voraz no es óptimo ({len(subopt)} jornadas): diferencia media = {statistics.mean(subopt):.4f} Mill.")
```

Experimento 1: 10000 jornadas aleatorias (19 perfiles de audiencia distintos)
 Jornadas en las que el voraz alcanza el óptimo: 88.26 %
 Diferencia absoluta (Mill.): media = 0.00133 | desv. típica = 0.00392 | varianza = 1.54e-05 | máximo = 0.02500

Diferencia relativa (%) : media = 0.02060 | desv. típica = 0.06071 |
varianza = 3.69e-03 | máximo = 0.39758
Cuando el voraz no es óptimo (1174 jornadas): diferencia media = 0.0114 Mill.

```
[33]: # --- Experimento 2: calendario completo (38 jornadas Round Robin) con todos
      ↪ los algoritmos ---
import time
import pandas as pd

random.seed(2024)
calendario_estudio = generar_calendario_robin(equipos)
algoritmos = ["Voraz", "Recocido simulado", "Programación dinámica"]
audiencia = {a: [] for a in algoritmos}
tiempo = {a: 0.0 for a in algoritmos}

for k, jornada in enumerate(calendario_estudio):
    t0 = time.perf_counter()
    audiencia["Voraz"].append(greedy_liga(jornada, audiencias_base,
    ↪ ponderaciones)[1])
    tiempo["Voraz"] += time.perf_counter() - t0

    t0 = time.perf_counter()
    audiencia["Recocido simulado"].append(
        recocido_simulado_liga(jornada, horarios, audiencias_base,
    ↪ ponderaciones, penalizaciones, semilla=k)[1])
    tiempo["Recocido simulado"] += time.perf_counter() - t0

    # La programación dinámica se ejecuta sin caché para medir su tiempo real
    t0 = time.perf_counter()
    audiencia["Programación dinámica"].append(
        dp_liga(jornada, horarios, audiencias_base, ponderaciones,
    ↪ penalizaciones)[1])
    tiempo["Programación dinámica"] += time.perf_counter() - t0

optimos = audiencia["Programación dinámica"]
total_optimo = sum(optimos)
filas = []
for a in algoritmos:
    difs = [o - v for o, v in zip(optimos, audiencia[a])]
    filas.append({
        "Algoritmo": a,
        "Audiencia total (M)": f"{sum(audiencia[a]):.3f}",
        "Pérdida vs. óptimo (M)": f"{total_optimo - sum(audiencia[a]):.3f}",
        "Pérdida (%)": f"{100 * (total_optimo - sum(audiencia[a])) /
    ↪ total_optimo:.3f}",
        "Jornadas con el óptimo": f"{sum(d < 1e-9 for d in difs)} /
    ↪ {len(difs)}",
```



```

        "Desv. típica dif. por jornada (M)": f"{statistics.pstdev(difs):.5f}",
        "Tiempo total": f"{tiempo[a] * 1e3:.1f} ms",
    })
# La fuerza bruta no se ejecuta: su tiempo se extrapola a partir de la
↳ estimación por jornada
filas.append({
    "Algoritmo": "Fuerza bruta (estimada)",
    "Audiencia total (M)": f"{total_optimo:.3f}",
    "Pérdida vs. óptimo (M)": "0.000",
    "Pérdida (%)": "0.000",
    "Jornadas con el óptimo": f"{len(optimos)} / {len(optimos)}",
    "Desv. típica dif. por jornada (M)": "0.00000",
    "Tiempo total": f" {horas * len(calendario_estudio) / 24:.1f} días",
})

print(f"\nExperimento 2: calendario completo de {len(calendario_estudio)}\n↳ jornadas")
display(pd.DataFrame(filas).set_index("Algoritmo"))

```

Experimento 2: calendario completo de 38 jornadas

	Audiencia total (M)	Pérdida vs. óptimo (M) \
Algoritmo		
Voraz	256.875	0.040
Recocido simulado	256.915	0.000
Programación dinámica	256.915	0.000
Fuerza bruta (estimada)	256.915	0.000

	Pérdida (%)	Jornadas con el óptimo \
Algoritmo		
Voraz	0.016	34 / 38
Recocido simulado	0.000	38 / 38
Programación dinámica	0.000	38 / 38
Fuerza bruta (estimada)	0.000	38 / 38

	Desv. típica dif. por jornada (M)	Tiempo total
Algoritmo		
Voraz	0.00307	0.2 ms
Recocido simulado	0.00000	361.0 ms
Programación dinámica	0.00000	1488.8 ms
Fuerza bruta (estimada)	0.00000	3.1 días

Estudio empírico del recocido simulado El recocido simulado es estocástico, así que su diferencia con el óptimo tiene dos fuentes de variabilidad: la jornada (los datos de entrada) y la semilla aleatoria (la ejecución). Se estudian ambas por separado: - **Experimento 3:** 2.000 jornadas aleatorias, una ejecución por jornada, comparando en las mismas jornadas el voraz y el recocido frente al óptimo de la programación dinámica. - **Experimento 4:** 200 ejecuciones con

semillas distintas sobre el contraejemplo (la jornada en la que el voraz falla), para medir la varianza propia del algoritmo. - **Tiempos:** coste real de cada algoritmo sobre la jornada del enunciado.

```
[34]: import time

# --- Experimento 3: voraz y recocido simulado frente al óptimo en las mismas
↳jornadas ---
random.seed(2025)
N_RS = 2_000
dif_vz, dif_rs, dif_rel_vz, dif_rel_rs = [], [], [], []

for k in range(N_RS):
    jornada = generar_jornada(equipos)
    clave = perfil(jornada)
    if clave not in cache_optimo:
        cache_optimo[clave] = dp_liga(jornada, horarios, audiencias_base,
↳ponderaciones, penalizaciones)[1]
    optimo = cache_optimo[clave]
    _, vz = greedy_liga(jornada, audiencias_base, ponderaciones)
    _, rs = recocido_simulado_liga(jornada, horarios, audiencias_base,
↳ponderaciones, penalizaciones, semilla=k)
    dif_vz.append(optimo - vz)
    dif_rs.append(optimo - rs)
    dif_rel_vz.append(100 * (optimo - vz) / optimo)
    dif_rel_rs.append(100 * (optimo - rs) / optimo)

def pct_optimo(difs):
    return 100 * sum(d < 1e-9 for d in difs) / len(difs)

print(f"Experimento 3: {N_RS} jornadas aleatorias")
print(f"Voraz    -> alcanza el óptimo en el {pct_optimo(dif_vz):.2f} % de las
↳jornadas")
resumen("  Diferencia absoluta (Mill.)", dif_vz)
resumen("  Diferencia relativa (%)    ", dif_rel_vz)
print(f"Recocido -> alcanza el óptimo en el {pct_optimo(dif_rs):.2f} % de las
↳jornadas")
resumen("  Diferencia absoluta (Mill.)", dif_rs)
resumen("  Diferencia relativa (%)    ", dif_rel_rs)
```

Experimento 3: 2000 jornadas aleatorias

Voraz -> alcanza el óptimo en el 88.85 % de las jornadas

Diferencia absoluta (Mill.): media = 0.00128 | desv. típica = 0.00387 |
varianza = 1.50e-05 | máximo = 0.02500

Diferencia relativa (%) : media = 0.01972 | desv. típica = 0.05982 |
varianza = 3.58e-03 | máximo = 0.39758

Recocido -> alcanza el óptimo en el 99.55 % de las jornadas

Diferencia absoluta (Mill.): media = 0.00003 | desv. típica = 0.00051 |
varianza = 2.61e-07 | máximo = 0.01000

Diferencia relativa (%) : media = 0.00050 | desv. típica = 0.00777 |
varianza = 6.03e-05 | máximo = 0.15372

```
[35]: # --- Experimento 4: varianza debida a la semilla, sobre el contraejemplo ---
N_SEMILLAS = 200
res_ce = [recocido_simulado_liga(contraejemplo, horarios, audiencias_base,
    ↪ponderaciones, penalizaciones, semilla=s)[1]
    for s in range(N_SEMILLAS)]
dif_ce = [aud_dp - r for r in res_ce]
print(f"\nExperimento 4: {N_SEMILLAS} ejecuciones del recocido sobre el
    ↪contraejemplo (óptimo = {aud_dp:.4f} Mill., voraz = {aud_voraz:.4f} Mill.)")
print(f"Recocido -> alcanza el óptimo en el {pct_optimo(dif_ce):.2f} % de las
    ↪ejecuciones")
resumen(" Diferencia absoluta (Mill.)", dif_ce)

# --- Tiempos de ejecución sobre la jornada del enunciado ---
def cronometrar(funcion, repeticiones):
    t0 = time.perf_counter()
    for _ in range(repeticiones):
        funcion()
    return (time.perf_counter() - t0) / repeticiones

t_vz = cronometrar(lambda: greedy_liga(partidos, audiencias_base,
    ↪ponderaciones), 1000)
t_rs = cronometrar(lambda: recocido_simulado_liga(partidos, horarios,
    ↪audiencias_base, ponderaciones, penalizaciones, semilla=0), 20)
t_dp = cronometrar(lambda: dp_liga(partidos, horarios, audiencias_base,
    ↪ponderaciones, penalizaciones), 3)
print(f"\nTiempo medio por jornada -> voraz: {t_vz * 1e3:.3f} ms | recocido
    ↪simulado: {t_rs * 1e3:.1f} ms | "
    f"programación dinámica: {t_dp * 1e3:.0f} ms | fuerza bruta (estimada):
    ↪{horas:.1f} h")
```

Experimento 4: 200 ejecuciones del recocido sobre el contraejemplo (óptimo = 6.3175 Mill., voraz = 6.3075 Mill.)

Recocido -> alcanza el óptimo en el 99.50 % de las ejecuciones

Diferencia absoluta (Mill.): media = 0.00002 | desv. típica = 0.00035 |
varianza = 1.24e-07 | máximo = 0.00500

Tiempo medio por jornada -> voraz: 0.002 ms | recocido simulado: 9.6 ms |
programación dinámica: 39 ms | fuerza bruta (estimada): 1.9 h

Sensibilidad de la heurística a los parámetros La heurística voraz asume implícitamente que las coincidencias nunca compensan. Para ver hasta qué punto depende eso de los datos del enunciado, repetimos el estudio con 50 jornadas de audiencias base aleatorias (uniformes entre 0,47 y 2,0 M) con (a) la tabla de penalizaciones del enunciado y (b) una tabla de penalizaciones más

suave (5 % por coincidencia, con mínimo del 55 %). En ambos casos se compara también el recocido simulado, que sí explora las coincidencias.

```
[36]: random.seed(7)
penal_suaves = {k: max(0.55, 1 - 0.05 * k) for k in range(10)}
res_enunciado, res_suaves = [], []
rs_enunciado, rs_suaves = [], []

for k in range(50):
    audiencias_rand = {f"P{i}": random.uniform(0.47, 2.0) for i in range(10)}
    partidos_rand = [{"partido": f"P{i}", "categorias": f"P{i}"} for i in
    ↪range(10)]
    _, voraz = greedy_liga(partidos_rand, audiencias_rand, ponderaciones)
    opt_a = dp_liga(partidos_rand, horarios, audiencias_rand, ponderaciones,
    ↪penalizaciones)[1]
    opt_b = dp_liga(partidos_rand, horarios, audiencias_rand, ponderaciones,
    ↪penal_suaves)[1]
    rs_a = recocido_simulado_liga(partidos_rand, horarios, audiencias_rand,
    ↪ponderaciones, penalizaciones, semilla=k)[1]
    rs_b = recocido_simulado_liga(partidos_rand, horarios, audiencias_rand,
    ↪ponderaciones, penal_suaves, semilla=k)[1]
    res_enunciado.append(100 * (opt_a - voraz) / opt_a)
    res_suaves.append(100 * (opt_b - voraz) / opt_b)
    rs_enunciado.append(100 * (opt_a - rs_a) / opt_a)
    rs_suaves.append(100 * (opt_b - rs_b) / opt_b)

resumen("Voraz, penalizaciones del enunciado, dif. relativa (%)",
    ↪res_enunciado)
resumen("Voraz, penalizaciones suaves, dif. relativa (%)", res_suaves)
resumen("Recocido, penalizaciones del enunciado, dif. relativa (%)",
    ↪rs_enunciado)
resumen("Recocido, penalizaciones suaves, dif. relativa (%)", rs_suaves)
```

```
Voraz, penalizaciones del enunciado, dif. relativa (%): media = 0.00000 |
desv. típica = 0.00000 | varianza = 1.97e-28 | máximo = 0.00000
Voraz, penalizaciones suaves, dif. relativa (%): media = 9.00163 |
desv. típica = 0.91481 | varianza = 8.37e-01 | máximo = 11.02134
Recocido, penalizaciones del enunciado, dif. relativa (%): media = 0.00000 |
desv. típica = 0.00000 | varianza = 1.09e-28 | máximo = 0.00000
Recocido, penalizaciones suaves, dif. relativa (%): media = 0.00158 |
desv. típica = 0.00794 | varianza = 6.30e-05 | máximo = 0.05394
```

1.1.4 Conclusiones del estudio empírico

Distancia al óptimo del voraz. Con los datos del enunciado, la heurística voraz queda muy cerca del óptimo, pero **no siempre lo alcanza**: - En 10.000 jornadas aleatorias obtiene el óptimo en el 88,26 % de los casos. En el resto se queda por debajo en 0,0114 millones de espectadores de media. - Diferencia absoluta: media de 0,0013 M, desviación típica de 0,0039 M (varianza

$1,5 \cdot 10^{-5}$) y máximo de 0,025 M. En términos relativos, media del 0,02 %, desviación típica del 0,06 % y máximo del 0,40 %. - La distribución de la diferencia es muy asimétrica: casi toda la masa está en 0 y el resto en unos pocos valores discretos, porque solo hay 19 perfiles de audiencia distintos y la diferencia aparece en 5 de ellos (jornadas con varios partidos de audiencia base parecida, donde compensa dejar vacía una franja de coeficiente bajo y juntar dos partidos en una franja alta). - En un calendario completo de 38 jornadas, el voraz pierde 0,040 M de 256,915 M (0,016 %) y no alcanza el óptimo en 4 de las 38 jornadas. Sobre ese mismo calendario, el recocido simulado obtiene el óptimo en todas las jornadas en 0,33 s, la programación dinámica en 1,4 s y la fuerza bruta necesitaría unos 3 días (estimado).

Distancia al óptimo del recocido simulado. En el experimento 3 (2.000 jornadas aleatorias, comparando ambas heurísticas sobre las mismas jornadas), el recocido alcanza el óptimo en el 99,55 % de los casos, frente al 88,85 % del voraz: - Diferencia absoluta: media de 0,00003 M y varianza $2,6 \cdot 10^{-7}$, unas 60 veces menor que la del voraz ($1,5 \cdot 10^{-5}$). La diferencia relativa media baja del 0,020 % al 0,0005 %. - Variabilidad propia del algoritmo: en 200 ejecuciones con semillas distintas sobre el contraejemplo, encuentra la solución óptima con coincidencias en el 99,5 % de los casos (varianza de la diferencia $1,2 \cdot 10^{-7}$). Es decir, el resultado apenas depende de la semilla. - Coste: unos 9 ms por jornada, frente a 0,002 ms del voraz y 38 ms de la programación dinámica (y unas 2 horas estimadas para la fuerza bruta).

Sensibilidad. La calidad del voraz depende de los parámetros del problema. Con audiencias aleatorias continuas y las penalizaciones del enunciado, ambas heurísticas coinciden con el óptimo en todos los casos probados. Esto no contradice el 12 % de jornadas en las que el voraz falla con los datos reales: juntar dos partidos solo compensa la penalización del 25 % cuando varios partidos tienen audiencias base iguales o muy parecidas. Con las categorías A/B/C esos empates son muy frecuentes (por ejemplo, varios partidos B-B de 0,9 M), mientras que con audiencias aleatorias continuas casi nunca se producen. Con penalizaciones suaves, en cambio, el voraz se queda un 9,0 % por debajo de media (desviación típica de 0,91 puntos, máximo del 11 %), porque entonces las coincidencias sí compensan y el voraz las ignora. El recocido simulado, que sí explora las coincidencias, se mantiene a un 0,002 % del óptimo de media (máximo del 0,05 %).

Conclusiones. 1. El algoritmo voraz es una buena heurística con los datos del enunciado, pero no se puede afirmar que garantice el óptimo global (el contraejemplo lo demuestra), y su calidad se degrada mucho si las penalizaciones son menos severas. 2. La programación dinámica sobre subconjuntos obtiene el óptimo exacto en una fracción de segundo ($O(m \cdot 3^n)$ frente a $O(n^2 m^n)$), por lo que para este tamaño es el algoritmo recomendado y sirve de referencia para medir las heurísticas. 3. El recocido simulado ofrece el mejor compromiso para escalar el problema: coste $O(K \cdot n)$, no exponencial, y una distancia al óptimo prácticamente nula y con varianza muy baja, incluso cuando las coincidencias son parte de la solución óptima.

Enumera las referencias que has utilizado (si ha sido necesario) para llevar a cabo el trabajo

1. Easton, K., Nemhauser, G., & Trick, M. (2001). The Traveling Tournament Problem Description and Benchmarks (CP 2001)
2. FLM System. (s. f.). Round robin scheduling. <https://www.flmsystem.com/blog/round-robin-scheduling>
3. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
4. Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

1.1.5 Declaración de uso de Inteligencia Artificial (AID)

Con el fin de garantizar la transparencia y la integridad académica, se declara el uso de herramientas de Inteligencia Artificial como apoyo durante la elaboración de este trabajo. La responsabilidad final sobre el contenido, las decisiones adoptadas y las conclusiones presentadas corresponde exclusivamente a los autores.

1. Herramienta utilizada

- ChatGPT (OpenAI, GPT-5.5)
- Gemini
- Claude (Anthropic)

2. Propósito del uso

La herramienta se utilizó como apoyo para:

- Mejora de los algoritmos utilizados.
- Revisión de las correcciones del profesor e implementación del recocido simulado y de su estudio empírico.
- Resolver dudas conceptuales relacionadas con los algoritmos voraces y la planificación *Round Robin*.
- Mejorar la redacción y corrección ortográfica del documento.
- Obtener sugerencias sobre la estructura y presentación del trabajo.
- Ayudar en la búsqueda y el formateo de referencias bibliográficas en formato APA.

3. Prompts (instrucciones) utilizados

Algunos ejemplos de las consultas realizadas fueron:

- Como puedo mejorar el siguiente algoritmo para un algoritmo de optimización.
- “Explica el funcionamiento de un algoritmo voraz aplicado a la planificación de torneos Round Robin.”
- “Corrige la redacción y las faltas ortográficas de este apartado.”
- “Revisa la calidad académica del notebook y sugiere posibles mejoras.”

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

En este seminario se asume un escenario determinista basado en 10 partidos estáticos. Sin embargo, la organización real del calendario de La Liga es un problema de optimización altamente complejo (catalogado como NP-Hard). Para avanzar en el estudio de este problema y acercarlo a un entorno de producción real, se proponen las siguientes líneas de investigación y variaciones:

1. En la práctica, la asignación de horarios no depende únicamente de la audiencia televisiva, sino de la disponibilidad de recursos logísticos. Se deben modelar restricciones relacionadas con la seguridad ciudadana. Por ejemplo, hay que evitar que dos equipos de la misma ciudad (ej. Real Madrid y Atlético de Madrid) jueguen como locales el mismo día, ya que los recursos policiales y de movilidad urbana no pueden cubrir ambos eventos simultáneamente.
2. El algoritmo actual no tiene en cuenta el estado previo de los equipos. Una variación fundamental consistiría en añadir una variable temporal que conecte las jornadas entre sí, incorporando las competiciones europeas o la Copa del Rey. Esto obligaría al algoritmo a respetar ventanas de descanso obligatorias (mínimo de 72 horas). Si un equipo juega el jueves en Europa, matemáticamente su espacio de soluciones válido para La Liga se reduciría, impidiendo

que se le asigne el horario de máxima audiencia del sábado, y forzando al modelo a buscar un subóptimo global.

3. Si se aumenta el tamaño del problema para incluir simultáneamente la primera y la segunda división, el volumen pasaría a ser de más de 21 partidos por fin de semana. En este volumen, el solapamiento de partidos dejaría de ser evitable y la penalización por coincidencia se activaría obligatoriamente.

Para soportar este incremento drástico en el tamaño del espacio de búsqueda, el algoritmo voraz perdería su eficacia (ignora las coincidencias, que serían inevitables) y la programación dinámica dejaría de ser viable por su coste exponencial. El siguiente paso en el estudio requeriría implementar otro tipo de arquitecturas para abordar la explosión combinatoria. El recocido simulado implementado en este trabajo es un primer paso en esa dirección, ya que su coste $O(K \cdot n)$ no depende exponencialmente del número de partidos. Además, se proponen otros métodos de búsqueda local estocástica, es decir, algoritmos que exploran el espacio de soluciones aplicando modificaciones mediante un factor de azar matemático muy controlado.

Dentro de esta familia, se destacan los algoritmos genéticos. En lugar de construir un único calendario paso a paso, este sistema generaría una “población” inicial de múltiples calendarios completos y simularía un proceso evolutivo. Seleccionando los calendarios que devuelvan mayor audiencia, el algoritmo combinaría sus estructuras e introduciría pequeños cambios aleatorios en los horarios de partidos concretos. Al iterar este ciclo durante miles de generaciones, el sistema descartaría las asignaciones ineficientes y convergería de forma orgánica hacia el calendario global óptimo.

Referencia del óptimo para escalar el problema. El algoritmo de programación dinámica de este trabajo ($O(m \cdot 3^n)$) proporciona el óptimo exacto para jornadas de hasta unos 15 partidos, por lo que puede usarse como referencia para medir la distancia al óptimo (y su varianza) de las metaheurísticas anteriores (recocido simulado, búsqueda tabú, algoritmos genéticos). A partir de unos 20 partidos el coste exponencial lo hace inviable y habría que comparar las metaheurísticas entre sí o frente a cotas superiores.